ПЕРВАЯ ЛАБОРАТОРНАЯ РАБОТА

Как работает калькулятор и каждый его тест

Подробный разбор C# / .NET 8 / Windows Forms / MSTest

Внутри: запуск программы, настройка формы, арифметика, преобразование строк, сообщения об ошибках, все 26 автоматических тестов и способы воспроизведения старых ситуаций.

Уточнение про * и /

В выпадающем списке формы есть только +, -, × и ÷. Ввести туда * или / нельзя: у списка установлен режим DropDownList. Тесты этих символов вызывают Calculator напрямую. Они проверяют добавленную возможность программного интерфейса модуля, а не ошибку выбора операции в окне.

По исходному набору операций отсутствие * и / само по себе не является дефектом. В прежнем объяснении эти ситуации были названы ошибками слишком широко. На защите их следует обозначать как дополнительные проверки расширенного контракта метода.

Основа документа - реальные исходники репозитория SoftwareTesting. Код не изменён при подготовке разбора. Примеры результатов со знаком запятой предполагают культуру ru-RU.

Содержание

1. Что делает приложение и где проходит граница 4

2. Проекты, зависимости и запуск 5

3. Program.cs: от запуска до окна 6

4. Calculator.cs: полный арифметический модуль 7

5. decimal, литералы и исключения 8

6. TryCalculate: весь путь обработки запроса 9

7. CurrentCulture, InvariantCulture и G29 10

8. TryReadNumber: оба пути разбора числа 11

9. Разбор группировки и регулярных выражений 12

10. TryParse, нормализация и пределы ввода 13

11. Form1.cs: полный код поведения окна 14

12. Обработчики кнопок и изменений 15

13. Designer: поля, ресурсы, создание элементов 16

14. Designer: операция и два поля чисел 17

15. Designer: кнопки, ответ и сообщение 18

16. Designer: настройки самой формы 19

17. Как MSTest находит и выполняет тесты 20

18.1. Модульные тесты: 1-2 из 10 21

18.2. Модульные тесты: 3-4 из 10 22

18.3. Модульные тесты: 5-6 из 10 23

18.4. Модульные тесты: 7-8 из 10 24

18.5. Модульные тесты: 9-10 из 10 25

19. Общий метод Check интеграционных тестов 26

20.1. Интеграционные тесты: 1-2 из 9 27

20.2. Интеграционные тесты: 3-4 из 9 28

20.3. Интеграционные тесты: 5-6 из 9 29

20.4. Интеграционные тесты: 7-8 из 9 30

20.5. Интеграционные тесты: 9-9 из 9 31

21. Как тесты находят элементы формы 32

22. RunOnSta: поток, ожидание и передача ошибки 33

23.1. Тест формы: расчёт через форму 34

23.2. Тест формы: ошибка после предыдущего расчёта 35

23.3. Тест формы: очистка формы 36

23.4. Тест формы: изменение первого числа 37

23.5. Тест формы: изменение операции 38

23.6. Тест формы: обработка enter 39

23.7. Тест формы: поле ответа только для чтения 40

24. Что покрыто, а что тесты не доказывают 41

25. История ситуаций: точные формулировки 42

26. Как воспроизвести старые состояния 43

27. Как объяснить работу на защите 44

28. Словарь конструкций C# из проекта 45

29. Источники и как сверять объяснение с кодом 46

1. Что делает приложение и где проходит граница

Калькулятор выполняет одну выбранную операцию над двумя числами. Пользователь вводит первое и второе число, выбирает действие и нажимает «Вычислить» или Enter. Это не разбор целого выражения: строка 2+3*4, скобки и приоритет операций здесь не реализованы.

Часть	Принимает	Возвращает или меняет
Form1	Действия и текст из элементов окна	Поля результата и ошибки
CalculatorSession	Две строки и строку операции	bool, строку результата, строку ошибки
Calculator	Два decimal и строку операции	decimal либо исключение

Один пример от начала до конца

1. Пользователь вводит 2,5 и 4, выбирает ×. В TextBox лежат строки «2,5» и «4», а не числовые значения.

2. Обработчик кнопки передаёт строки в CalculatorSession.TryCalculate. TryReadNumber читает из них decimal 2.5 и 4.

3. Сессия вызывает Calculator.Calculate(2.5m, 4m, "×"). Арифметический модуль возвращает decimal со значением 10.

4. Сессия форматирует значение в строку «10» и возвращает true. Форма помещает эту строку в поле результата, а сообщение ошибки очищает.

Какие значения может передать пользователь

В поля чисел можно вставить и неверный текст, поэтому проверка чисел обязательна. Операция выбирается из закрытого списка, поэтому обычный пользователь не может передать «?», «*» или «/» как операцию через это окно. Прямой вызов публичного метода из теста такими ограничениями формы не связан.

Зачем три части

Арифметику можно проверять без окна и без строк. Обработку ввода можно проверять без окна, но вместе с арифметикой. Форму проверяют последней, когда нужно убедиться, что элементы и события правильно связывают эти части. Такое разделение облегчает поиск причины ошибки.

2. Проекты, зависимости и запуск

Основной проект: SoftwareTesting.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net8.0-windows</TargetFramework>
    <Nullable>enable</Nullable>
    <UseWindowsForms>true</UseWindowsForms>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <Compile Remove="Tests\**\*.cs" />
  </ItemGroup>

</Project>
```

Sdk выбирает стандартные правила сборки .NET. OutputType=WinExe создаёт оконное приложение. TargetFramework=net8.0-windows задаёт .NET 8 с Windows API. UseWindowsForms подключает WinForms. Это Windows-приложение; одинаковый запуск на других ОС этот проект не обещает.

Nullable=enable включает анализ возможных null. Он предупреждает о подозрительных местах при компиляции, но не делает проверки за программу. ImplicitUsings добавляет типовые using автоматически: поэтому не в каждом файле явно написаны using System или using System.Windows.Forms.

Compile Remove исключает исходники Tests из основного проекта. По умолчанию SDK ищет .cs и во вложенных каталогах. Без исключения тестовые классы попали бы и в приложение, которому зависимости MSTest не подключены.

Тестовый проект: Tests/SoftwareTesting.Tests.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0-windows</TargetFramework>
    <UseWindowsForms>true</UseWindowsForms>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.11.1" />
    <PackageReference Include="MSTest.TestAdapter" Version="3.6.4" />
    <PackageReference Include="MSTest.TestFramework" Version="3.6.4" />
    <ProjectReference Include="..\SoftwareTesting.csproj" />
  </ItemGroup>
</Project>
```

IsTestProject отмечает назначение проекта. Microsoft.NET.Test.Sdk обеспечивает инфраструктуру запуска. MSTest.TestFramework содержит атрибуты и Assert. MSTest.TestAdapter помогает средствам тестирования обнаруживать и выполнять MSTest-тесты. ProjectReference даёт тестам доступ к публичным классам приложения.

3. Program.cs: от запуска до окна

```
namespace SoftwareTesting
{
    internal static class Program
    {
        /// <summary>
        ///  The main entry point for the application.
        /// </summary>
        [STAThread]
        static void Main()
        {
            // To customize application configuration such as set high DPI settings or default font,
            // see https://aka.ms/applicationconfiguration.
            ApplicationConfiguration.Initialize();
            Application.Run(new Form1());
        }
    }
}
```

namespace SoftwareTesting объединяет классы проекта в пространство имён. internal ограничивает доступ к Program текущей сборкой. static означает, что объект Program не создаётся: метод Main вызывается как точка входа приложения.

[STAThread] задаёт модель STA для главного потока. Она нужна для корректной работы ряда Windows-компонентов, используемых интерфейсом. Этот атрибут действует на запуск приложения; поток тестового хоста он автоматически не перенастраивает.

ApplicationConfiguration.Initialize() применяет настройки приложения WinForms, подготовленные .NET для проекта: например, параметры отображения и шрифта. Здесь нет вычислений калькулятора.

new Form1() создаёт окно и вызывает его конструктор. Application.Run запускает цикл обработки сообщений Windows для главной формы. Благодаря этому окно реагирует на клики, ввод и перерисовку. При закрытии главной формы этот цикл завершается.

Комментарии // и XML-комментарий /// поясняют код человеку и инструментам. Сами по себе они не исполняются.

Как запустить

В Visual Studio открывай SoftwareTesting.sln: в решении видны и приложение, и отдельный тестовый проект. Назначь SoftwareTesting запускаемым проектом, собери решение и нажми Ctrl+F5. Для тестов открой «Тест → Обозреватель тестов».

```
dotnet run --project SoftwareTesting.csproj
dotnet test SoftwareTesting.sln
```

Команды выполняются в папке репозитория. Первое действие запускает окно, второе собирает и запускает тесты. Для работы нужны Windows и совместимый .NET SDK с необходимыми средствами .NET 8. Сборка проекта и его запуск - разные действия.

4. Calculator.cs: полный арифметический модуль

```
namespace SoftwareTesting
{
    public static class Calculator
    {
        public static decimal Calculate(decimal firstNumber, decimal secondNumber, string operation)
        {
            return operation switch
            {
                "+" => firstNumber + secondNumber,
                "-" => firstNumber - secondNumber,
                "×" or "*" => firstNumber * secondNumber,
                "÷" or "/" => secondNumber == 0
                    ? throw new DivideByZeroException("На ноль делить нельзя.")
                    : firstNumber / secondNumber,
                _ => throw new ArgumentException("Неизвестная операция.", nameof(operation))
            };
        }
    }
}
```

Заголовок класса и метода

public позволяет обратиться к классу из другого проекта, в том числе из тестового. static позволяет писать Calculator.Calculate(...), не создавая new Calculator(). Метод не хранит результаты прошлых вызовов.

Первое decimal задаёт тип результата. firstNumber и secondNumber содержат уже прочитанные числа. string operation содержит обозначение действия. Поля формы и CurrentCulture этому методу не нужны.

Как выбирается действие

return operation switch - выражение выбора. Каждая ветка имеет вид «образец => результат». Для "+" возвращается сумма; для "-" - разность. Знак => здесь отделяет ветку switch от выражения; это не объявление отдельной лямбда-функции.

"×" or "*" означает: подходит любая из двух строк. Аналогично "÷" or "/". Это два обозначения одной операции, а не отдельные арифметические действия. В форме используются × и ÷, а * и / проверяются прямыми вызовами метода.

В ветке деления применяется условное выражение: условие ? значение_если_да : значение_если_нет. Если secondNumber == 0, выбрасывается исключение. Иначе выполняется деление.

_ - ветка для всех остальных строк. ArgumentException сообщает о неподдерживаемом аргументе. nameof(operation) возвращает строку «operation» и указывает, какой параметр вызвал проблему.

5. decimal, литералы и исключения

Почему decimal

decimal хранит десятичное число с ограниченной точностью, примерно 28-29 значащих цифр. Для обычных десятичных дробей вроде 0.1 он удобен тем, что многие такие значения представляет точно. У double другое, двоичное представление; некоторые десятичные дроби в нём являются приближениями.

decimal не даёт бесконечной точности. Например, 1m / 3m получается конечным приближением. Максимальное положительное значение decimal: 79228162514264337593543950335. Результат за допустимыми пределами приводит к OverflowException.

Почему написано 1.5m

```
decimal fraction = 1.5m;
decimal whole = 2m;
// 1.5 без m - литерал типа double, 2 без m - типа int.
```

m или M в конце числового литерала задаёт decimal. Это суффикс языка C#, он не часть значения и не единица измерения. В исходнике десятичный разделитель - точка. Ввод пользователя «1,5» является строкой и разбирается отдельно.

Целое число int часто может быть неявно преобразовано в decimal, поэтому m у целых аргументов не всегда обязателен. В тестах он делает тип ожидаемого и фактического значения очевидным. double в decimal не преобразуется неявно.

Три ожидаемых исключения

Ситуация	Исключение модуля	Действие сессии
Деление на 0	DivideByZeroException	«На ноль делить нельзя.»
Слишком большое значение результата	OverflowException	«Результат слишком велик.»
Неизвестная операция	ArgumentException	«Выберите операцию.»

throw прерывает нормальное выполнение метода. Calculator не возвращает вместо ошибки условный ноль: ноль сам может быть правильным ответом. CalculatorSession перехватывает ожидаемые исключения и превращает их в сообщение.

Проверка неверного текста выполняется раньше арифметики через TryParse. Для такого ввода сессия возвращает false без необходимости вызывать Calculator.

6. TryCalculate: весь путь обработки запроса

```
public static bool TryCalculate(string firstText, string secondText, string operation,
    out string result, out string error)
{
    result = string.Empty;
    error = string.Empty;

    if (!TryReadNumber(firstText, out decimal firstNumber) ||
        !TryReadNumber(secondText, out decimal secondNumber))
    {
        error = "Введите два числа.";
        return false;
    }

    try
    {
        decimal value = Calculator.Calculate(firstNumber, secondNumber, operation);
        result = value.ToString("G29", CultureInfo.CurrentCulture);
        return true;
    }
    catch (DivideByZeroException)
    {
        error = "На ноль делить нельзя.";
    }
    catch (OverflowException)
    {
        error = "Результат слишком велик.";
    }
    catch (ArgumentException)
    {
        error = "Выберите операцию.";
    }

    return false;
}
```

Метод принимает firstText, secondText и operation. Возвращаемый bool показывает успех. Параметры out result и out error позволяют дополнительно вернуть два строковых значения. Вызывающий код получает все три результата одного вызова.

result = string.Empty и error = string.Empty задают исходное состояние при каждом вызове. string.Empty равен пустой строке "". Это не null: строка существует, но не содержит символов.

В условии ! отрицает результат TryReadNumber. Оператор || означает «или» и сокращает вычисление: если первое число уже неверное, второе в этой строке может не проверяться. После отказа сразу выполняется return false. В сообщение не включается номер неверного поля: оно общее для обоих чисел.

Если оба числа прочитаны, выполнение входит в try. Calculator.Calculate возвращает decimal, ToString готовит текст ответа, затем return true завершает метод. Если Calculate выбрасывает одно из указанных исключений, выполняется подходящий catch.

Каждый catch задаёт сообщение. После блока обработки общий return false сообщает о неуспехе; result остаётся пустым. Здесь не перехватываются вообще все возможные ошибки, например NullReferenceException: такой catch в коде отсутствует.

7. CurrentCulture, InvariantCulture и G29

CurrentCulture

CultureInfo описывает региональные правила: разделитель дробной части, группировку разрядов, представление дат и другие форматы. CultureInfo.CurrentCulture - текущая культура выполнения, используемая операциями чтения и форматирования, если код к ней обращается.

В ru-RU обычная запись дроби - «1,5», в en-US - «1.5». Математическое значение одинаковое. Культура влияет на перевод между текстом и числом, а не на результат 1.5m * 2m.

```
CultureInfo.CurrentCulture = CultureInfo.GetCultureInfo("ru-RU");
```

Тесты явно устанавливают ru-RU, чтобы ожидание «1235,5» было воспроизводимым. В Program.cs нет такого принудительного присваивания, поэтому запущенное приложение обычно использует культуру окружения. Отсюда запятая в результате не гарантирована на любом компьютере.

InvariantCulture

InvariantCulture - стабильные правила, не зависящие от выбранного языка Windows. В используемом числовом формате дробная часть отделяется точкой. Сессия применяет эту культуру после того, как сама привела запись с группировкой к единому виду.

Формат результата

```
result = value.ToString("G29", CultureInfo.CurrentCulture);
```

ToString превращает число в текст. G - общий формат, 29 - максимальное число значащих цифр при таком форматировании. Это не требование вывести 29 знаков после запятой. Для decimal явная точность в G позволяет не сохранять ненужные конечные нули в рассматриваемых случаях: 3.0m отображается как «3».

CurrentCulture выбирает разделитель в готовой строке. Формат G29 не увеличивает точность уже вычисленного числа. Для некоторых величин общий формат может использовать экспоненциальную запись; отдельного требования всегда показывать только обычную запись в коде нет.

8. TryReadNumber: оба пути разбора числа

```
private static bool TryReadNumber(string text, out decimal number)
{
    string input = text.Trim();

    if (input.Contains(',') && input.Contains('.'))
    {
        bool commaIsGroup = input.LastIndexOf(',') < input.LastIndexOf('.');
        string pattern = commaIsGroup
            ? @"^[+-]?[0-9]{1,3}(?:,[0-9]{3})+\.[0-9]+$"
            : @"^[+-]?[0-9]{1,3}(?:\.[0-9]{3})+,[0-9]+$";

        if (!Regex.IsMatch(input, pattern))
        {
            number = 0;
            return false;
        }

        string normalizedGroup = input.Replace(commaIsGroup ? "," : ".", string.Empty)
            .Replace(",", ".");
        return decimal.TryParse(normalizedGroup,
            NumberStyles.AllowLeadingSign | NumberStyles.AllowDecimalPoint,
            CultureInfo.InvariantCulture, out number);
    }

    string separator = CultureInfo.CurrentCulture.NumberFormat.NumberDecimalSeparator;
    string normalized = input.Replace(",", separator).Replace(".", separator);
    return decimal.TryParse(normalized, NumberStyles.Number,
        CultureInfo.CurrentCulture, out number);
}
```

private означает, что TryReadNumber вызывается только внутри CalculatorSession. Его проверяют косвенно через публичный TryCalculate. Возвращаемый bool сообщает, удалось ли чтение, out decimal number содержит прочитанное значение.

Trim удаляет пробельные символы по краям строки. Внутренние пробелы эта операция не удаляет. Сам метод предполагает, что text не null: при null вызов Trim приведёт к исключению. Тексты TextBox обычно передаются как непустые либо пустые строки.

Если присутствуют и запятая, и точка, используется отдельный путь со строгой проверкой групп разрядов. В остальных случаях точка и запятая заменяются на текущий десятичный разделитель, затем вызывается decimal.TryParse.

9. Разбор группировки и регулярных выражений

Как определяется роль разделителей

Для записи с обоими символами код сравнивает последние позиции: LastIndexOf возвращает индекс последнего вхождения. Если последняя запятая расположена левее последней точки, запятая считается разделителем групп, а точка - дробной части.

Вход	Выбранная роль	После нормализации
1,234.5	Запятая - группы, точка - дробь	1234.5
1.234,5	Точка - группы, запятая - дробь	1234.5

Шаблон для записи 1,234.5

```
^[+-]?[0-9]{1,3}(?:,[0-9]{3})+\.[0-9]+$
```

Фрагмент	Смысл
^	Начало строки
[+-]?	Необязательный плюс или минус
[0-9]{1,3}	Первая группа: от одной до трёх цифр
(?:,[0-9]{3})+	Одна или больше групп: запятая и ровно три цифры
\.	Буквальная точка, а не любой символ
[0-9]+	Одна или больше цифр дробной части
$	Конец строки в правилах .NET Regex

Второй шаблон устроен аналогично, но точка и запятая меняются ролями. ?: означает группу без сохранения отдельного захваченного значения. @ перед строкой C# позволяет записать обратную косую черту без удвоения; сами правила Regex от этого не меняются.

Так, 12,34.5 не соответствует шаблону: после запятой только две цифры вместо трёх. Код присваивает number = 0 и возвращает false. Этот ноль - служебное значение out при отказе, его нельзя принимать за успешно прочитанное число.

10. TryParse, нормализация и пределы ввода

Путь с двумя разделителями

Replace удаляет разделители групп. Следующий Replace заменяет дробную запятую на точку. Например, 1.234,5 становится 1234,5, затем 1234.5. После проверки шаблона TryParse получает уже понятную запись для InvariantCulture.

NumberStyles.AllowLeadingSign разрешает знак перед числом. AllowDecimalPoint разрешает десятичный разделитель. Побитовое | объединяет флаги. Оно не означает логическое «или» || и не выбирает только один из флагов.

Путь с одним видом разделителя или без него

Из CurrentCulture берётся NumberDecimalSeparator. При ru-RU это запятая. Поэтому вход 1.5 превращается в 1,5, а 1,5 остаётся таким же. Далее decimal.TryParse читает строку с NumberStyles.Number и CurrentCulture.

NumberStyles.Number допускает не только дробную часть, но и другие элементы стандартной числовой записи, включая некоторые пробельные символы, группировку и конечный знак. Значит, этот путь не равен строгому шаблону «только цифры и одна запятая». Текущий набор тестов не фиксирует все такие варианты.

Как читать результат TryParse

```
bool ok = decimal.TryParse(text, styles, culture, out number);
```

При успехе ok равен true, а number содержит decimal. При обычном неверном числовом тексте или выходе за диапазон ok равен false. TryParse не выбрасывает FormatException из-за строки «abc». Если слишком большое число введено уже в поле, сессия покажет «Введите два числа.», а «Результат слишком велик.» относится к переполнению во время вычисления.

Неоднозначная запись

Одинокий разделитель в «1,234» или «1.234» здесь нормализуется как десятичный. Это значение 1.234, а не автоматически 1234. Поддержка тысяч в явном виде реализована для записей с обоими символами, например 1,234.5. Не стоит обещать, что программа понимает любой национальный формат числа.

11. Form1.cs: полный код поведения окна

```
namespace SoftwareTesting
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void CalculateButton_Click(object sender, EventArgs e)
        {
            if (CalculatorSession.TryCalculate(firstNumberTextBox.Text,
                secondNumberTextBox.Text, operationComboBox.Text,
                out string result, out string error))
            {
                resultTextBox.Text = result;
                errorLabel.Text = string.Empty;
            }
            else
            {
                resultTextBox.Clear();
                errorLabel.Text = error;
            }
        }

        private void ClearButton_Click(object sender, EventArgs e)
        {
            firstNumberTextBox.Clear();
            secondNumberTextBox.Clear();
            resultTextBox.Clear();
            errorLabel.Text = string.Empty;
            operationComboBox.SelectedIndex = 0;
            firstNumberTextBox.Focus();
        }

        private void InputChanged(object sender, EventArgs e)
        {
            resultTextBox.Clear();
            errorLabel.Text = string.Empty;
        }
    }
}
```

partial означает, что один класс описан в нескольких файлах. Form1.cs содержит поведение, Form1.Designer.cs - создание и настройку элементов. Наследование : Form даёт классу возможности окна Windows Forms.

Конструктор Form1 вызывается при new Form1(). InitializeComponent создаёт элементы, задаёт их свойства и подписывает обработчики событий. Без этой инициализации поля элементов не указывали бы на готовые объекты.

12. Обработчики кнопок и изменений

CalculateButton_Click

sender - источник события, EventArgs e - его данные. Сигнатура нужна для подписки на Click, хотя сами параметры внутри этого обработчика не используются. Метод читает .Text у двух полей и списка, затем вызывает TryCalculate.

При true в resultTextBox.Text записывается результат и стирается старая ошибка. При false результат очищается, а errorLabel.Text получает сообщение. Поэтому число и сообщение об ошибке не должны показываться как два действующих результата одновременно.

ClearButton_Click

Clear у TextBox задаёт пустой текст. Затем очищается сообщение, SelectedIndex = 0 выбирает первый элемент списка - плюс, а Focus пытается вернуть ввод в первое поле. Focus возвращает bool, но этот обработчик его не проверяет.

Изменение текста через Clear тоже может вызвать TextChanged. Поэтому InputChanged может выполниться во время очистки. Повторное присваивание пустых значений безопасно: обработчик только очищает ответ и ошибку и не запускает новый расчёт.

InputChanged

Этот метод подписан на TextChanged обоих числовых полей и SelectedIndexChanged списка. Как только пользователь меняет данные, прежний ответ больше не соответствует текущему вводу. Обработчик убирает его сразу, ещё до нового нажатия «Вычислить».

Новое значение автоматически не вычисляется. Чтобы получить ответ для изменённых данных, нужно снова нажать кнопку или Enter. Сообщение ошибки тоже убирается, потому что оно относилось к предыдущему вводу.

Почему события вызываются и в тестах

Присваивание Text или SelectedItem из кода также вызывает соответствующие события WinForms при изменении значения. Поэтому тест может воспроизвести очистку старого ответа, присвоив новое значение, без физического набора с клавиатуры.

13. Designer: поля, ресурсы, создание элементов

```
namespace SoftwareTesting
{
    partial class Form1
    {
        private System.ComponentModel.IContainer components = null;
        private Label firstNumberLabel;
        private Label secondNumberLabel;
        private Label operationLabel;
        private Label resultLabel;
        private Label errorLabel;
        private TextBox firstNumberTextBox;
        private TextBox secondNumberTextBox;
        private TextBox resultTextBox;
        private ComboBox operationComboBox;
        private Button calculateButton;
        private Button clearButton;

        /// <summary>
        ///  Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            firstNumberLabel = new Label();
            secondNumberLabel = new Label();
            operationLabel = new Label();
            resultLabel = new Label();
            errorLabel = new Label();
            firstNumberTextBox = new TextBox();
            secondNumberTextBox = new TextBox();
            resultTextBox = new TextBox();
            operationComboBox = new ComboBox();
            calculateButton = new Button();
            clearButton = new Button();
            SuspendLayout();
```

Label - подпись, TextBox - текстовое поле, ComboBox - список, Button - кнопка. Приватные поля сохраняют ссылки на элементы, чтобы код обработчиков мог их менять. Это объекты интерфейса, а не числа калькулятора.

components - контейнер компонентов, созданных по обычной схеме Designer. В этой версии он остаётся null. Dispose(bool disposing) освобождает содержимое контейнера, если он есть и допускается освобождение управляемых ресурсов. Затем base.Dispose передаёт освобождение базовому классу, включая ресурсы окна и дочерних элементов.

InitializeComponent сначала создаёт все объекты через new. SuspendLayout временно приостанавливает расчёт компоновки, пока меняются свойства. Поэтому при настройке не нужно заново раскладывать элементы после каждого присваивания.

Это начало Designer; продолжение приведено в следующих трёх разделах. Скобки закрываются в последнем фрагменте.

14. Designer: операция и два поля чисел

```
operationLabel.AutoSize = true;
operationLabel.Location = new Point(24, 25);
operationLabel.Text = "Операция";
operationComboBox.DropDownStyle = ComboBoxStyle.DropDownList;
operationComboBox.Name = "operationComboBox";
operationComboBox.Items.AddRange(new object[] { "+", "-", "×", "÷" });
operationComboBox.Location = new Point(24, 48);
operationComboBox.Size = new Size(252, 28);
operationComboBox.SelectedIndex = 0;
operationComboBox.SelectedIndexChanged += InputChanged;

firstNumberLabel.AutoSize = true;
firstNumberLabel.Location = new Point(24, 90);
firstNumberLabel.Text = "Первое число";
firstNumberTextBox.Location = new Point(24, 113);
firstNumberTextBox.Name = "firstNumberTextBox";
firstNumberTextBox.Size = new Size(252, 27);
firstNumberTextBox.TextChanged += InputChanged;

secondNumberLabel.AutoSize = true;
secondNumberLabel.Location = new Point(24, 154);
secondNumberLabel.Text = "Второе число";
secondNumberTextBox.Location = new Point(24, 177);
secondNumberTextBox.Name = "secondNumberTextBox";
secondNumberTextBox.Size = new Size(252, 27);
secondNumberTextBox.TextChanged += InputChanged;
```

AutoSize у подписи подбирает размер по тексту. Location задаёт координаты относительно родительского контейнера: Point(x, y). Size задаёт ширину и высоту. Text - видимая надпись, Name - имя элемента для поиска из кода и тестов; это разные свойства.

Почему нельзя выбрать * или /

DropDownStyle = DropDownList запрещает свободное редактирование списка. AddRange добавляет ровно четыре варианта: +, -, ×, ÷. SelectedIndex = 0 выбирает плюс. Значит, недоступность * и / в прежнем Calculator не ломала существующий выбор в форме.

SelectedIndexChanged += InputChanged подписывает обработчик на изменение выбранного пункта. += здесь добавляет обработчик события, а не складывает значения. Подписка устанавливается после начального выбора плюса.

У обоих полей TextChanged подписан на тот же InputChanged. По этой причине и ввод в первом поле, и ввод во втором должны убирать прежний результат. Автотест в текущем наборе отдельно меняет только первое поле; второй вариант логически следует из подписки, но отдельным тестом не зафиксирован.

15. Designer: кнопки, ответ и сообщение

```
calculateButton.Location = new Point(24, 222);
calculateButton.Name = "calculateButton";
calculateButton.Size = new Size(120, 36);
calculateButton.Text = "Вычислить";
calculateButton.UseVisualStyleBackColor = true;
calculateButton.Click += CalculateButton_Click;
clearButton.Location = new Point(156, 222);
clearButton.Name = "clearButton";
clearButton.Size = new Size(120, 36);
clearButton.Text = "Очистить";
clearButton.UseVisualStyleBackColor = true;
clearButton.Click += ClearButton_Click;

resultLabel.AutoSize = true;
resultLabel.Location = new Point(24, 275);
resultLabel.Text = "Результат";
resultTextBox.Location = new Point(24, 298);
resultTextBox.Name = "resultTextBox";
resultTextBox.ReadOnly = true;
resultTextBox.Size = new Size(252, 27);
errorLabel.AutoSize = true;
errorLabel.ForeColor = Color.Firebrick;
errorLabel.Location = new Point(24, 338);
errorLabel.Name = "errorLabel";
errorLabel.MaximumSize = new Size(252, 0);
```

Click += связывает кнопку с методом из Form1.cs. Сама надпись «Вычислить» не запускает арифметику: действие появляется благодаря подписке на Click. UseVisualStyleBackColor разрешает оформление фона кнопки средствами темы Windows.

resultTextBox.ReadOnly = true запрещает пользователю редактировать ответ. Код программы по-прежнему может присвоить Text или вызвать Clear. Это не то же самое, что Enabled=false: поле остаётся доступным для выделения и копирования текста.

errorLabel.ForeColor задаёт цвет сообщения Firebrick. MaximumSize = new Size(252, 0) ограничивает ширину; ноль по высоте означает отсутствие заданного максимума. Вместе с AutoSize это позволяет подписи увеличивать высоту при переносе текста.

Настройка MaximumSize не доказывает, что любой текст при любом масштабе поместится в окно. Отдельная визуальная проверка масштабирования этим кодом и существующими тестами не заменяется.

16. Designer: настройки самой формы

```
            AcceptButton = calculateButton;
            AutoScaleMode = AutoScaleMode.Font;
            ClientSize = new Size(300, 390);
            Controls.Add(operationLabel);
            Controls.Add(operationComboBox);
            Controls.Add(firstNumberLabel);
            Controls.Add(firstNumberTextBox);
            Controls.Add(secondNumberLabel);
            Controls.Add(secondNumberTextBox);
            Controls.Add(calculateButton);
            Controls.Add(clearButton);
            Controls.Add(resultLabel);
            Controls.Add(resultTextBox);
            Controls.Add(errorLabel);
            FormBorderStyle = FormBorderStyle.FixedSingle;
            MaximizeBox = false;
            Name = "Form1";
            StartPosition = FormStartPosition.CenterScreen;
            Text = "Арифметический калькулятор";
            ResumeLayout(false);
            PerformLayout();
        }
    }
}
```

AcceptButton = calculateButton назначает кнопку по умолчанию для Enter, когда клавиша не обработана текущим элементом иначе. AutoScaleMode.Font задаёт масштабирование относительно шрифта. ClientSize - размер внутренней области окна, без рамки и заголовка.

Controls.Add добавляет каждый элемент в коллекцию дочерних элементов формы. Одного new Button() недостаточно для его появления в окне. Здесь элементы добавлены прямо на форму; поиск теста с true умеет проходить и вложенные контейнеры.

FixedSingle задаёт фиксированную рамку. MaximizeBox=false убирает возможность штатного разворачивания кнопкой. StartPosition.CenterScreen задаёт начальное положение. Name - программное имя формы, Text - видимый заголовок.

ResumeLayout(false) возобновляет обработку компоновки без немедленного принудительного расчёта этим вызовом. PerformLayout выполняет компоновку после настройки. Последние фигурные скобки закрывают InitializeComponent, класс и namespace.

Что помнить для защиты

Designer содержит обычный C#-код создания интерфейса. Он отделён от обработчиков для удобства работы конструктора Visual Studio. Поведение кнопки определяется подпиской на событие, а состав списка - AddRange. Поэтому доказательство ограниченного выбора операции находится именно в этом файле.

17. Как MSTest находит и выполняет тесты

```
using System.Globalization;
using Microsoft.VisualStudio.TestTools.UnitTesting;
using SoftwareTesting;

namespace SoftwareTestingTests
{
    [TestClass]
    public class CalculatorTests
    {
        [TestInitialize]
        public void SetCulture()
        {
            CultureInfo.CurrentCulture =
                CultureInfo.GetCultureInfo("ru-RU");
        }
        // Ниже находятся отдельные [TestMethod].
    }
}
```

Это сокращённый каркас; отдельные методы приведены далее. using System.Globalization подключает имена CultureInfo. using MSTest подключает Assert и атрибуты. using SoftwareTesting даёт короткий доступ к классам приложения.

[TestClass] отмечает класс с тестами. [TestMethod] отмечает отдельный тест, который появится в Обозревателе тестов. Метод SetCulture помечен [TestInitialize], поэтому MSTest вызывает его перед каждым тестом данного класса. В CalculatorSessionTests есть такой же метод.

У CalculatorFormTests нет SetCulture: культура устанавливается внутри рабочего STA-потока в RunOnSta. Это нужно, потому что именно там выполняется действие с формой. Всего обнаруживается 10 + 9 + 7 = 26 тестовых методов; Check и RunOnSta отдельными тестами не являются.

Что такое Assert и схема AAA

Arrange - подготовить данные и состояние. Act - вызвать проверяемое действие. Assert - сравнить итог с ожиданием. В коротком тесте Act и Assert находятся в одной строке, но логические шаги остаются теми же.

Assert.AreEqual(expected, actual) сравнивает ожидаемое и фактическое. Assert.IsTrue требует true. Assert.ThrowsException<T> требует исключение заданного типа от переданного действия. Если нужного исключения нет, такой тест не проходит.

Зелёный тест означает совпадение с ожиданием, включая корректную реакцию на ошибочный ввод. Деление на ноль может быть причиной успешного теста, если ожидается именно обработка такой ситуации.

18.1. Модульные тесты: 1-2 из 10

Addition - Сложение

```
[TestMethod]
public void Addition()
{
    Assert.AreEqual(5m, Calculator.Calculate(2m, 3m, "+"));
}
```

Передаются 2m, 3m и "+". Calculate должен вернуть decimal 5. Assert сравнивает два числовых значения, формат вывода здесь не участвует.

Это базовый положительный сценарий. Он подтверждает один пример сложения, но не все сочетания чисел.

SubtractionOfNegativeNumbers - Вычитание отрицательных

```
[TestMethod]
public void SubtractionOfNegativeNumbers()
{
    Assert.AreEqual(-2m, Calculator.Calculate(-5m, -3m, "-"));
}
```

Вычисляется -5 - (-3). Вычитание отрицательного равносильно прибавлению 3, поэтому ожидается -2. Все аргументы и ожидание заданы как decimal.

Тест полезен для проверки знаков: неправильный порядок вычитания или ошибочная обработка минуса даст другое значение.

18.2. Модульные тесты: 3-4 из 10

MultiplicationOfFractions - Умножение дроби

```
[TestMethod]
public void MultiplicationOfFractions()
{
    Assert.AreEqual(3m, Calculator.Calculate(1.5m, 2m, "×"));
}
```

Передаются 1.5m, 2m и "×". Ожидание - 3m. Суффикс m задаёт decimal, а не double. Ни строки «1,5», ни CurrentCulture для этой арифметики не нужны.

Проверяется операция над уже готовыми числами. Поддержка запятой во вводе относится к интеграционным тестам.

Division - Деление с дробным ответом

```
[TestMethod]
public void Division()
{
    Assert.AreEqual(2.5m, Calculator.Calculate(5m, 2m, "÷"));
}
```

Вычисляется 5m / 2m, ожидается 2.5m. Такой пример обнаружил бы ошибочное целочисленное деление с ответом 2.

Проверяется числовое значение 2.5; то, будет ли оно показано как «2,5», этот тест не устанавливает.

18.3. Модульные тесты: 5-6 из 10

DivisionByZero - Деление на ноль

```
[TestMethod]
public void DivisionByZero()
{
    Assert.ThrowsException<DivideByZeroException>(() => Calculator.Calculate(5m, 0m, "÷"));
}
```

ThrowsException получает отложенное действие () => Calculator.Calculate(...). При выполнении действия должен возникнуть DivideByZeroException. Если вернётся число, тест завершится неуспешно.

Проверяется тип исключения. Текст исключения и пользовательская надпись на форме здесь не сравниваются.

UnknownOperation - Неизвестная операция

```
[TestMethod]
public void UnknownOperation()
{
    Assert.ThrowsException<ArgumentException>(() => Calculator.Calculate(1m, 2m, "?"));
}
```

В метод напрямую передаётся "?". Ветка _ должна выбросить ArgumentException. Символ «?» отсутствует в закрытом списке формы.

Это проверка защиты публичного метода от неподдерживаемого аргумента, доступная прямому вызывающему коду. Через текущий список пользователь такую операцию не выбирает.

18.4. Модульные тесты: 7-8 из 10

Overflow - Переполнение при сложении

```
[TestMethod]
public void Overflow()
{
    Assert.ThrowsException<OverflowException>(() => Calculator.Calculate(decimal.MaxValue, 1m, "+"));
}
```

decimal.MaxValue - наибольшее decimal. К нему прибавляется 1m. Представить точную сумму этим типом нельзя, поэтому ожидается OverflowException.

Входные числа допустимы по отдельности. Ошибка появляется при вычислении результата, а не при чтении строки.

MultiplicationOverflow - Переполнение при умножении

```
[TestMethod]
public void MultiplicationOverflow()
{
    Assert.ThrowsException<OverflowException>(() => Calculator.Calculate(decimal.MaxValue, 2m, "×"));
}
```

decimal.MaxValue умножается на 2m с операцией "×". Ожидается OverflowException. Это другая арифметическая ветка, чем в Overflow.

Здесь нет ручной проверки диапазона в Calculator: соответствующее исключение возникает при операции decimal.

18.5. Модульные тесты: 9-10 из 10

MultiplicationWithAsciiSign - Дополнительное обозначение *

```
[TestMethod]
public void MultiplicationWithAsciiSign()
{
    Assert.AreEqual(12m, Calculator.Calculate(3m, 4m, "*"));
}
```

Calculator вызывается напрямую с 3m, 4m и "*". Ожидается 12m, потому что текущий метод принимает оба обозначения умножения.

В форме * недоступен. Старое отсутствие этой возможности можно называть несоответствием только расширенному контракту метода, где поддержка * уже требуется. Для исходного выбора × это не пользовательский дефект.

DivisionWithAsciiSign - Дополнительное обозначение /

```
[TestMethod]
public void DivisionWithAsciiSign()
{
    Assert.AreEqual(3m, Calculator.Calculate(12m, 4m, "/"));
}
```

Calculator вызывается напрямую с 12m, 4m и "/". Ожидается 3m. Текущая ветка switch принимает / вместе с ÷.

В форме / недоступен. Тест фиксирует расширение программного интерфейса и защищает его от последующей потери; он не воспроизводит неверный выбор через окно.

19. Общий метод Check интеграционных тестов

```
private static void Check(string first, string second, string operation, string result, string error)
{
    bool success = CalculatorSession.TryCalculate(first, second, operation,
        out string actualResult, out string actualError);
    Assert.AreEqual(error.Length == 0, success);
    Assert.AreEqual(result, actualResult);
    Assert.AreEqual(error, actualError);
}
```

Check получает пять ожидаемых строк: first и second - ввод, operation - действие, result - ожидаемый ответ, error - ожидаемая ошибка. Он нужен, чтобы в девяти сценариях не повторять один и тот же вызов и три проверки.

Сначала TryCalculate реально выполняет разбор и расчёт. actualResult и actualError - фактические значения, записанные через out. success - фактический bool. Названия actual помогают отличить результат программы от ожидаемых result и error.

error.Length == 0 вычисляет ожидаемую успешность. В этих сценариях пустая ожидаемая ошибка означает true, непустая - false. Первый Assert сравнивает этот bool с success. Второй сравнивает текст ответа, третий - текст ошибки.

Например, Check("abc", "2", "+", "", "Введите два числа.") одновременно требует false, пустой ответ и правильное сообщение. Если метод вернёт true с тем же сообщением, первая проверка обнаружит противоречие.

Assert не собирает автоматически все расхождения: при первом неуспешном утверждении выполнение теста обычно прерывается. Если сравнение bool прошло, это ещё не гарантирует прохождение двух строковых проверок.

Почему это интеграционный уровень

Реально взаимодействуют преобразование текста, числовой модуль и форматирование. Подмен и моков Calculator в проекте нет. Кнопки и окно при этом не создаются. Отдельно private TryReadNumber не вызывается: его работа проверяется через TryCalculate.

20.1. Интеграционные тесты: 1-2 из 9

FractionWithComma - Дробь с запятой

```
[TestMethod]
public void FractionWithComma()
{
    Check("1,5", "2", "×", "3", "");
}
```

"1,5", "2", "×" → true, «3», пустая ошибка. В ru-RU запятая уже совпадает с текущим десятичным разделителем. Числа передаются в Calculator, а G29 убирает ненужный конечный ноль.

Связывает чтение строки, умножение и форматирование. Не проверяет такой же ввод при каждой возможной культуре.

FractionWithDot - Дробь с точкой

```
[TestMethod]
public void FractionWithDot()
{
    Check("1.5", "2", "×", "3", "");
}
```

"1.5", "2", "×" → true, «3», пустая ошибка. Для ru-RU точка заменяется запятой до TryParse. Результат должен совпасть с предыдущим сценарием.

Отдельный тест важен: обычное чтение только по ru-RU без нормализации могло бы отвергнуть точку или прочитать её иначе.

20.2. Интеграционные тесты: 3-4 из 9

EmptyInput - Пустое поле

```
[TestMethod]
public void EmptyInput()
{
    Check("", "2", "+", "", "Введите два числа.");
}
```

Первая строка пустая, вторая равна «2». TryReadNumber для первой возвращает false. Ожидаются false, пустой ответ и «Введите два числа.».

Проверяется пустота первого поля. Отдельного теста пустого второго поля или двух пустых полей в текущем наборе нет.

DivisionByZero - Сообщение о делении на ноль

```
[TestMethod]
public void DivisionByZero()
{
    Check("5", "0", "÷", "", "На ноль делить нельзя.");
}
```

Строки «5» и «0» успешно читаются. Calculator выбрасывает DivideByZeroException; catch сессии задаёт сообщение «На ноль делить нельзя.». Ожидаются false и пустой ответ.

В отличие от одноимённого модульного теста здесь ожидается нормальный возврат метода с сообщением, а не исключение наружу.

20.3. Интеграционные тесты: 5-6 из 9

Overflow - Сообщение о переполнении суммы

```
[TestMethod]
public void Overflow()
{
    Check(decimal.MaxValue.ToString(), "1", "+", "", "Результат слишком велик.");
}
```

MaxValue.ToString() делает допустимую строку максимального числа. Прибавление 1 вызывает OverflowException уже после успешного чтения. Ожидаются false, пустой ответ и «Результат слишком велик.».

SetCulture задаёт ru-RU перед тестом. Здесь не проверяется ввод строки, которая сама больше MaxValue: это иной путь отказа.

MultiplicationOverflow - Сообщение о переполнении произведения

```
[TestMethod]
public void MultiplicationOverflow()
{
    Check(decimal.MaxValue.ToString(), "2", "×", "", "Результат слишком велик.");
}
```

Максимальное decimal передаётся строкой, второе число - «2», операция - «×». После разбора умножение переполняется. Check требует false и сообщение «Результат слишком велик.».

Проверяет, что одинаковая обработка переполнения действует и для другой операции, а результат не остаётся числом.

20.4. Интеграционные тесты: 7-8 из 9

InvalidInput - Нечисловой текст

```
[TestMethod]
public void InvalidInput()
{
    Check("abc", "2", "+", "", "Введите два числа.");
}
```

Первое поле содержит «abc», второе - «2». Чтение первой строки не удаётся; до арифметики выполнение не доходит. Ожидаются false, пустой ответ и «Введите два числа.».

Сценарий отличается от пустоты: поле заполнено, но его содержимое не является допустимой числовой записью.

GroupedNumberWithCommaThousands - Тысячи через запятую

```
[TestMethod]
public void GroupedNumberWithCommaThousands()
{
    Check("1,234.5", "1", "+", "1235,5", "");
}
```

«1,234.5» означает 1234.5. Сначала проверяется группировка, затем запятая удаляется. После прибавления 1 получается 1235.5, в ru-RU это «1235,5». Ошибка должна быть пустой, success - true.

Такой текст можно вставить в обычное поле формы. В отличие от символов * и / у списка операций, ввод здесь действительно доступен пользователю.

20.5. Интеграционные тесты: 9-9 из 9

GroupedNumberWithDotThousands - Тысячи через точку

```
[TestMethod]
public void GroupedNumberWithDotThousands()
{
    Check("1.234,5", "1", "+", "1235,5", "");
}
```

«1.234,5» сначала превращается в «1234.5». Затем выполняется сложение с 1 и форматирование ответа в «1235,5». Проверяются true, ответ и пустая ошибка.

Сценарий проверяет другой порядок разделителей. Два успешных примера не подтверждают обработку любого неправильного сочетания точек и запятых.

21. Как тесты находят элементы формы

```
private static T Control<T>(Form form, string name) where T : Control
{
    return (T)form.Controls.Find(name, true).Single();
}
```

Это обобщённый вспомогательный метод, а не отдельный тест. T означает тип нужного элемента. При Control<TextBox>(...) вместо T используется TextBox, при Control<Button>(...) - Button. where T : Control ограничивает выбор типами элементов интерфейса.

form.Controls.Find(name, true) ищет элементы по свойству Name. true включает поиск во вложенных дочерних коллекциях. Возвращается набор найденных элементов. Single требует ровно один результат: отсутствие элемента или несколько совпадений приведут к исключению и провалу теста.

(T) приводит найденный элемент к ожидаемому типу. Если по данному имени найдётся, например, Label вместо TextBox, такое приведение не выполнится. Тест не продолжит молча работу с неверным элементом.

```
Control<TextBox>(form, "firstNumberTextBox").Text = "2";
Control<Button>(form, "calculateButton").PerformClick();
```

Первая строка находит поле по имени и присваивает текст. Вторая находит кнопку и программно инициирует её действие. Тесты не обращаются к private-полям Form1 напрямую: они идут через публичную коллекцию Controls.

Зачем using и Show

using Form1 form = new Form1() гарантирует вызов Dispose при выходе из области, в том числе после неуспешного Assert. Это using-объявление для освобождения ресурсов, а не using в начале файла для пространства имён.

Show делает форму видимой и создаёт необходимое состояние окна. В этих тестах он используется до PerformClick: кнопка должна быть доступна и допускающей действие. Автоматическое открытие формы может быть кратковременным, поскольку по окончании теста она освобождается.

22. RunOnSta: поток, ожидание и передача ошибки

```
private static void RunOnSta(Action action)
{
    Exception? error = null;
    Thread thread = new Thread(() =>
    {
        try
        {
            CultureInfo.CurrentCulture = CultureInfo.GetCultureInfo("ru-RU");
            action();
        }
        catch (Exception exception)
        {
            error = exception;
        }
    });
    thread.SetApartmentState(ApartmentState.STA);
    thread.Start();
    thread.Join();

    if (error != null)
    {
        throw error;
    }
}
```

Action - действие без аргументов и без возвращаемого значения. RunOnSta получает от теста лямбду с созданием формы, вводом и Assert, затем выполняет её в другом потоке.

Exception? error = null объявляет ссылку, которая может быть null. Пока ошибки нет, значение остаётся null. new Thread создаёт поток с переданным телом; сам конструктор поток ещё не запускает.

Внутри потока сначала задаётся ru-RU, затем вызывается action(). Любое обычное Exception от действия, включая неуспешный Assert, сохраняется в error. Это связывает результат рабочего потока с результатом теста.

SetApartmentState(STA) задаёт модель потока до Start. Start начинает выполнение, Join блокирует вызывающий поток до завершения рабочего. Поэтому проверка не заканчивается раньше обработки формы и становится доступным записанный результат.

После Join сохранённое исключение выбрасывается в потоке теста. Без такого переноса исключение в отдельном потоке не было бы обычным понятным результатом данного MSTest-метода.

У текущего помощника нет тайм-аута: если действие зависнет, Join может ждать бесконечно. throw error заново выбрасывает объект и может изменить вид стека; исходный контекст точнее сохранял бы ExceptionDispatchInfo. Эти особенности не проверяются самим набором тестов.

23.1. Тест формы: расчёт через форму

```
[TestMethod]
public void CalculationThroughForm()
{
    RunOnSta(() =>
    {
        using Form1 form = new Form1();
        form.Show();
        Control<ComboBox>(form, "operationComboBox").SelectedItem = "×";
        Control<TextBox>(form, "firstNumberTextBox").Text = "2,5";
        Control<TextBox>(form, "secondNumberTextBox").Text = "4";
        Control<Button>(form, "calculateButton").PerformClick();
        Assert.AreEqual("10", Control<TextBox>(form, "resultTextBox").Text);
    });
}
```

RunOnSta запускает весь сценарий в подготовленном STA-потоке. using создаёт форму с гарантированным освобождением, Show открывает её.

SelectedItem = "×" выбирает существующий пункт списка. Затем Text двух полей получает «2,5» и «4». При изменении ввода могут сработать InputChanged и очистка ответа.

PerformClick вызывает действие кнопки «Вычислить». Её Click ведёт в CalculateButton_Click, далее в сессию и Calculator. Assert сравнивает текст поля результата с «10».

Проверяется конкретный расчёт через реальные элементы формы. Сообщение ошибки отдельно не проверяется; остальные три операции через окно этим одним тестом не покрыты. Физическое движение мыши не имитируется.

23.2. Тест формы: ошибка после предыдущего расчёта

```
[TestMethod]
public void ErrorClearsPreviousResult()
{
    RunOnSta(() =>
    {
        using Form1 form = new Form1();
        form.Show();
        Control<TextBox>(form, "firstNumberTextBox").Text = "5";
        Control<TextBox>(form, "secondNumberTextBox").Text = "2";
        Control<Button>(form, "calculateButton").PerformClick();
        Control<ComboBox>(form, "operationComboBox").SelectedItem = "÷";
        Control<TextBox>(form, "secondNumberTextBox").Text = "0";
        Control<Button>(form, "calculateButton").PerformClick();
        Assert.AreEqual("", Control<TextBox>(form, "resultTextBox").Text);
        Assert.AreEqual("На ноль делить нельзя.", Control<Label>(form, "errorLabel").Text);
    });
}
```

В форме задаются 5 и 2. Первая кнопка вычисляет сумму с исходным плюсом. Затем операция меняется на ÷, второе число - на 0, и кнопка нажимается ещё раз.

Сессия должна вернуть отказ и сообщение. Последние Assert требуют пустое поле ответа и текст «На ноль делить нельзя.». Если надпись другая либо поле непустое, тест не проходит.

Тонкость текущей версии: смена операции и числа уже вызывает InputChanged и заранее убирает прежний ответ. Поэтому сценарий проверяет итог после ошибки, но не изолированно доказывает, что именно ветка else обработчика кнопки очистила непустой результат.

Кроме того, перед ошибкой нет отдельного Assert для первой суммы. Тест не следует описывать как доказательство всех промежуточных состояний. Отдельные тесты изменения числа и операции проверяют эти переходы точнее.

23.3. Тест формы: очистка формы

```
[TestMethod]
public void ClearRestoresInitialState()
{
    RunOnSta(() =>
    {
        using Form1 form = new Form1();
        form.Show();
        Control<TextBox>(form, "firstNumberTextBox").Text = "5";
        Control<TextBox>(form, "secondNumberTextBox").Text = "0";
        Control<ComboBox>(form, "operationComboBox").SelectedItem = "÷";
        Control<Button>(form, "calculateButton").PerformClick();
        Control<Button>(form, "clearButton").PerformClick();
        Assert.AreEqual("", Control<TextBox>(form, "firstNumberTextBox").Text);
        Assert.AreEqual("", Control<TextBox>(form, "secondNumberTextBox").Text);
        Assert.AreEqual("", Control<TextBox>(form, "resultTextBox").Text);
        Assert.AreEqual("", Control<Label>(form, "errorLabel").Text);
        Assert.AreEqual("+", Control<ComboBox>(form, "operationComboBox").Text);
    });
}
```

Вводятся 5 и 0, выбирается ÷ и нажимается «Вычислить». Так подготавливается состояние ошибки. Затем PerformClick вызывается у clearButton.

Пять проверок требуют: пустое первое поле, пустое второе поле, пустой ответ, пустая ошибка и текст операции "+". Это конкретное проверяемое определение начального состояния.

В реализации ClearButton_Click также вызывает Focus у первого поля. Этот тест фокус не проверяет. Он также не проверяет успешность подготовительного сообщения до нажатия «Очистить».

Пустой ответ после очистки здесь проверяется из состояния деления на ноль. Отдельного сценария «очистить уже непустой успешный ответ» в этом методе нет, хотя обработчик результата вызывает Clear безусловно.

23.4. Тест формы: изменение первого числа

```
[TestMethod]
public void EditingNumberClearsPreviousResult()
{
    RunOnSta(() =>
    {
        using Form1 form = new Form1();
        form.Show();
        Control<TextBox>(form, "firstNumberTextBox").Text = "2";
        Control<TextBox>(form, "secondNumberTextBox").Text = "3";
        Control<Button>(form, "calculateButton").PerformClick();
        Assert.AreEqual("5", Control<TextBox>(form, "resultTextBox").Text);
        Control<TextBox>(form, "firstNumberTextBox").Text = "10";
        Assert.AreEqual("", Control<TextBox>(form, "resultTextBox").Text);
    });
}
```

Вводятся 2 и 3, с исходным плюсом нажимается расчёт. Первый Assert требует «5»: этим подтверждается, что старый ответ действительно существовал.

Далее в firstNumberTextBox записывается «10». TextChanged синхронно вызывает InputChanged, а он очищает результат. Второй Assert требует пустую строку.

Нового нажатия «Вычислить» нет. Это существенная часть сценария: прежний ответ должен исчезнуть сразу после редактирования, а не после нового расчёта.

Проверяется первое поле. Изменение второго поля и очистка старой ошибки этим методом отдельно не проверяются. На второй TextBox в Designer подписан тот же обработчик, но это наблюдение по коду.

23.5. Тест формы: изменение операции

```
[TestMethod]
public void ChangingOperationClearsPreviousResult()
{
    RunOnSta(() =>
    {
        using Form1 form = new Form1();
        form.Show();
        Control<TextBox>(form, "firstNumberTextBox").Text = "2";
        Control<TextBox>(form, "secondNumberTextBox").Text = "3";
        Control<Button>(form, "calculateButton").PerformClick();
        Assert.AreEqual("5", Control<TextBox>(form, "resultTextBox").Text);
        Control<ComboBox>(form, "operationComboBox").SelectedItem = "×";
        Assert.AreEqual("", Control<TextBox>(form, "resultTextBox").Text);
    });
}
```

Подготавливается сумма 2 + 3, и первый Assert подтверждает ответ «5». Затем SelectedItem получает "×".

При смене пункта срабатывает SelectedIndexChanged. Подписанный InputChanged удаляет старый ответ. Последний Assert проверяет пустое поле.

Этот тест дополняет предыдущий: реализация могла подписать только TextChanged чисел и забыть подписку списка. Тогда изменение числа работало бы правильно, а этот сценарий обнаружил бы упущение.

Здесь не требуется сразу показать 6: автоматического повторного расчёта в приложении нет. Ожидаемое поведение - убрать устаревшее значение.

23.6. Тест формы: обработка enter

```
[TestMethod]
public void EnterStartsCalculation()
{
    RunOnSta(() =>
    {
        using Form1 form = new Form1();
        form.Show();
        Control<TextBox>(form, "firstNumberTextBox").Text = "2";
        Control<TextBox>(form, "secondNumberTextBox").Text = "3";
        Control<TextBox>(form, "secondNumberTextBox").Focus();
        System.Reflection.MethodInfo? processDialogKey = typeof(Form).GetMethod("ProcessDialogKey",
            System.Reflection.BindingFlags.Instance | System.Reflection.BindingFlags.NonPublic);
        bool handled = (bool)processDialogKey!.Invoke(form, new object[] { Keys.Enter })!;
        Assert.IsTrue(handled);
        Assert.AreEqual("5", Control<TextBox>(form, "resultTextBox").Text);
    });
}
```

После создания формы задаются 2 и 3. Focus пытается активировать второе поле. Затем через отражение ищется непубличный метод ProcessDialogKey у базового Form.

typeof(Form) возвращает описание типа. GetMethod с Instance | NonPublic ищет метод экземпляра, недоступный как обычный public-вызов. MethodInfo? допускает null, если метод не найден.

processDialogKey! снимает предупреждение компилятора о возможном null, но не выполняет проверку. Invoke вызывает найденный метод у form с массивом из одного аргумента Keys.Enter. Его возвращаемый object приводится к bool.

Assert.IsTrue(handled) требует, чтобы клавиша была обработана. Второй Assert требует сумму «5», значит действие кнопки привело к вычислению.

Это программный вызов обработки диалоговой клавиши, а не отправка физического нажатия через Windows. Успех Focus отдельно не проверяется. Тест зависит от наличия метода базовой библиотеки.

23.7. Тест формы: поле ответа только для чтения

```
[TestMethod]
public void ResultIsReadOnly()
{
    RunOnSta(() =>
    {
        using Form1 form = new Form1();
        form.Show();
        Control<TextBox>(form, "firstNumberTextBox").Text = "2";
        Control<TextBox>(form, "secondNumberTextBox").Text = "3";
        Control<Button>(form, "calculateButton").PerformClick();
        TextBox result = Control<TextBox>(form, "resultTextBox");
        Assert.AreEqual("5", result.Text);
        Assert.IsTrue(result.ReadOnly);
    });
}
```

В форме вычисляется 2 + 3. Затем поле результата сохраняется в переменную result, чтобы дважды не повторять его поиск.

Первый Assert требует текст «5». Второй Assert.IsTrue(result.ReadOnly) проверяет установленное свойство только для чтения.

ReadOnly относится к пользовательскому редактированию. Это не запрет присваивать result.Text из программы: иначе калькулятор вообще не смог бы показывать ответ.

Тест не пытается напечатать символы в поле и не проверяет операции буфера обмена. Его точное утверждение - свойство ReadOnly равно true после расчёта.

24. Что покрыто, а что тесты не доказывают

Уровень	Число методов	Проверяемая часть
Модульный	10	Арифметика и исключения Calculator
Интеграционный	9	Чтение строк, расчёт, ответ и сообщения
Форма	7	Элементы, события, итоговые состояния
Всего	26	26 отдельных [TestMethod]

Все эти методы существуют в текущем исходнике. Этот документ является разбором кода, а не новым протоколом прогона: фактический результат на твоём компьютере нужно показать запуском. В ранее выполненном прогоне проходили 26 из 26.

Заметные границы набора

1. Для арифметики есть конкретные примеры, а не все комбинации значений. Нет отдельного теста 1 / 3 и округления, вычитания с переполнением, нулевого множителя или каждой операции с положительными и отрицательными дробями.

2. Для сессии не выделены отдельные проверки пустого второго поля, неизвестной операции с корректными числами, неверной группировки, null и многих культур. Присутствие соответствующей ветки в исходнике не равно автоматическому покрытию этой ветки.

3. Тест формы проверяет умножение через окно, но не отдельный пользовательский сценарий каждой операции. Есть тест свойства ReadOnly и вызова обработки Enter, но нет полного физического ввода мышью и клавиатурой.

4. Читаемость, обрезание подписей при 100% и 150%, внешний вид ошибки и доступность интерфейса человеком в эти автоматические проверки не входят. Успешный Assert не оценивает изображение окна.

Почему тест на ошибку бывает зелёным

Ожидаемая ошибка - часть поведения программы. Если при 5 / 0 калькулятор корректно сообщает о запрете, проверка должна пройти. Красный тест означает расхождение фактического поведения с ожиданием, а не сам факт наличия неверного ввода.

25. История ситуаций: точные формулировки

Ситуация	Доступна через форму?	Как называть
* и / в Calculator	Нет, в списке только × и ÷	Расширение контракта прямого вызова
1,234.5 и 1.234,5	Да, текст можно вставить в поле	Добавленная поддержка форматов ввода
Старый ответ после редактирования	Да	Ошибка актуальности показанного результата

Тесты * и / не доказывают, что исходная форма умножала или делила неправильно. Старый метод понимал × и ÷, которые и передавала форма. Только после принятия дополнительного требования «метод обязан понимать ещё * и /» старое поведение не соответствует новому контракту.

Аналогично, необходимость принимать записи с группировкой должна следовать из принятого требования к вводу. Если требуются лишь обычные десятичные дроби, расширение форматов не нужно представлять как нарушение прежней спецификации.

Что это значит для требования о двух ситуациях на уровень

В истории есть по два дополнительных воспроизводимых случая на каждом уровне. Но модульные случаи * и / сами по себе не являются двумя доказанными дефектами исходного калькулятора с закрытым списком. Поэтому на защите нельзя без пояснения говорить, что они ломали пользовательское приложение.

Тесты деления на ноль и переполнения - реальные модульные проверки ошибочных условий, но текущая реализация обрабатывает их корректно. Наличие такого теста не означает, что в старой версии обязательно существовала соответствующая ошибка.

Если преподаватель требует именно два исторических дефекта исходной спецификации, это ограничение текущей демонстрации нужно учитывать: переименование расширения в «баг» доказательств не добавляет. Подготовленный здесь PDF не меняет ни требования, ни код, ни историю коммитов.

26. Как воспроизвести старые состояния

Команды запускаются из E:\study\SoftwareTesting. Перед переключением сохрани незакоммиченные изменения и закрой приложение. --detach открывает конкретный коммит без перехода на обычную именованную ветку. Для просмотра не нужно удалять файлы или выполнять git reset --hard.

Группа	Коммит с новыми проверками до реализации	Коммит реализации
CalculatorTests	daa0639	f7b08b7
CalculatorSessionTests	51dda16	9b84f39
CalculatorFormTests	63d4d3e	3a23efa

Прямой вызов Calculator с * и /

```
git switch --detach daa0639
dotnet test SoftwareTesting.sln --filter "FullyQualifiedName~SoftwareTestingTests.CalculatorTests"
git switch --detach f7b08b7
dotnet test SoftwareTesting.sln --filter "FullyQualifiedName~SoftwareTestingTests.CalculatorTests"
git switch master
```

В первом состоянии дополнительные тесты * и / ожидают результат, а модуль ещё отвергает эти строки. После реализации поддержка появляется. Обычным выбором операции в окне этот случай воспроизвести нельзя.

Два формата группировки

Переключись на 51dda16 и выполни команду тестов с фильтром FullyQualifiedName~CalculatorSessionTests. Затем повтори на 9b84f39. Для проверки в окне введи 1,234.5 и 1, выбери +; повтори с 1.234,5. В исправленном варианте ожидается числовое значение 1235.5, а знак в выводе зависит от культуры.

Старый ответ в окне

```
git switch --detach 63d4d3e
dotnet run --project SoftwareTesting.csproj
```

Вычисли 2 + 3 и измени первое число на 10: в старом варианте остаётся 5. Повтори расчёт и смени операцию на ×: ответ также остаётся. Закрой окно, перейди на 3a23efa, запусти снова и повтори действия: старый ответ исчезает. В конце закрой окно и выполни git switch master.

27. Как объяснить работу на защите

Последовательность показа

1. Сформулируй задачу: калькулятор с графическим интерфейсом, разделение на модули и проверка на трёх уровнях.

2. Запусти окно. Покажи обычную сумму, дробь с запятой и точкой, деление на ноль, неверный ввод, очистку и исчезновение старого ответа при изменении данных.

3. Открой Calculator, CalculatorSession и Form1. Объясни путь одного примера: строка → decimal → арифметика → строка → поле окна.

4. Запусти CalculatorTests. Разбери Addition и DivisionByZero. Объясни, почему в одном тесте ожидается число, а в другом исключение.

5. Запусти CalculatorSessionTests. Покажи FractionWithComma и Check. Объясни проверку bool, ответа и сообщения.

6. Запусти CalculatorFormTests. Покажи изменение числа после расчёта. Объясни RunOnSta, Show и PerformClick.

7. При необходимости воспроизведи старый ответ на историческом коммите и сравни с исправлением. Расширения * и / объясняй как прямые вызовы модуля.

8. Верни master и запусти весь набор. Назови фактическое число прошедших проверок, которое покажет запуск.

Короткий вывод своими словами

«Я разделил программу на арифметику, обработку текста и форму. Сначала проверил отдельные вычисления, затем их связь с вводом и форматированием, затем сценарии через элементы окна. Автоматический набор содержит 26 тестов. Он подтверждает проверенные случаи, но не доказывает отсутствие всех возможных ошибок».

Если спросят про * и /

«Вы правы: в окне их нет. Эти тесты вызывают публичный метод напрямую и фиксируют его дополнительную возможность. Исходный пользовательский выбор использует × и ÷; проблему в нём эти тесты не демонстрируют».

28. Словарь конструкций C# из проекта

Конструкция	Понятное объяснение
namespace	Группа имён типов. Помогает различать классы из разных библиотек.
class / static class	Описание объектов / класс со статическими членами без создания экземпляра.
public / private / internal	Доступ из другого проекта / только из класса / внутри сборки.
partial / : Form	Части одного класса в разных файлах / наследование от окна Form.
new / конструктор	Создание объекта / код начальной настройки объекта.
string / bool / decimal	Текст / true или false / десятичное число ограниченной точности.
return / out	Завершить метод и вернуть значение / дополнительно записать значение в выходной параметр.
if / else / ! / ||	Условие / альтернативная ветка / отрицание / сокращённое логическое «или».
switch / => / _ / or	Выбор ветки / результат ветки / всё остальное / несколько допустимых образцов.
try / catch / throw	Попытка действия / обработка исключения / выбрасывание исключения.
() => { ... }	Лямбда: передаваемое действие. Тело выполняется, когда действие вызывают.
using в начале файла	Подключение пространства имён для коротких имён типов.
using перед переменной	Освобождение IDisposable-объекта при выходе из области.
T / where T : Control	Параметр типа / ограничение на элементы интерфейса.
typeof / MethodInfo / Invoke	Описание типа / описание метода / его вызов через отражение.
? после ссылочного типа	Разрешение null для анализа nullable; не оператор условия в этой позиции.
! после выражения	Подавление предупреждения о null; не проверка во время выполнения.
(T) / (bool)	Приведение значения к ожидаемому типу.
[TestMethod] / [STAThread]	Атрибуты, которые читаются инфраструктурой; не обычные вызовы методов.

29. Источники и как сверять объяснение с кодом

Документ подготовлен по исходным файлам репозитория, прочитанным 20.10.2026. Приведённые методы взяты из текущих исходников; длинные строки кода могут быть показаны уменьшенным шрифтом. Сокращённый каркас тестового класса в разделе 17 отмечен отдельно.

Файл	Что разобрано
Program.cs	Точка входа целиком
SoftwareTesting.csproj	Настройки основного проекта целиком
Tests/SoftwareTesting.Tests.csproj	Настройки тестового проекта целиком
Calculator.cs	Класс и арифметический метод целиком
CalculatorSession.cs	TryCalculate и TryReadNumber целиком; using и назначение класса объяснены
Form1.cs	Класс и обработчики целиком
Form1.Designer.cs	Все строки файла в четырёх последовательных фрагментах
Tests/CalculatorTests.cs	Все 26 TestMethod, Check, Control<T>, RunOnSta; атрибуты и SetCulture объяснены

Путь к рабочей папке: E:\study\SoftwareTesting. Решение: SoftwareTesting.sln. Репозиторий: https://github.com/BananVovan34/SoftwareTesting. В старом LAB1.md число 22 относится к раннему прогону; текущий набор содержит 26 методов.

Контрольные суммы разобранных файлов

Короткие SHA-256 позволяют проверить, что файл не поменялся после подготовки PDF. Указаны первые 12 символов хеша исходных байтов. Это сведения об источнике, а не результат тестового прогона.

Файл	SHA-256, начало
Program.cs	d8a33466f831
Calculator.cs	8beec2347d8e
CalculatorSession.cs	e43e2e6eb16c
Form1.cs	2d240f7a8ad1
Form1.Designer.cs	8723283ad697
SoftwareTesting.csproj	a6a2f9e1fd60
Tests/SoftwareTesting.Tests.csproj	9139af05dd27
Tests/CalculatorTests.cs	2deacf0ba194